Vivekanand Education Society's Institute Of Technology
Department Of Information Technology
DSA mini Project
A.Y. 2024-25

Domain: Data Structures And Algorithm

Mentor Name: Rakhi Jadhav

Group Members:
Aaryan Shetye : 56
Shrey Pendurkar : 52
Ronit Sahoo : 54
Shakti Sankpal : 55

Title: Round Robin Scheduling



Table Of Contents

- Objectives of the Project
- Scope of the Project
- Data Structure & Concepts Used
- Algorithm Explanation
- Example
- Code
- Output Screenshots
- Conclusion
- References

Objectives Of The Project

Objective: To Understand The Application Of Circular Queue In Round Robin Scheduling.

- **Efficient Process Management:** Implement Round Robin scheduling using a circular queue to manage processes, optimizing CPU utilization.
- **Fair Time Allocation:** Ensure equal CPU time distribution for all processes by rotating them through fixed time slots (quantum), preventing process starvation.
- **Simulation & Metrics:** Simulate real-world scenarios to evaluate performance based on metrics like average waiting time, turnaround time, and context switching overhead.

Scope Of The Project

- **Implement Circular Queue:** Demonstrate its structure and use in managing resources efficiently.
- **Round Robin Scheduling:** Use a circular queue to fairly allocate CPU time to processes.
- **Simulation:** Simulate process scheduling with adjustable quantum and multiple processes.
- **Performance Metrics:** Analyze average waiting time, turnaround time, and context switch overhead.

Data Structures And Concepts Used

CIRCULAR QUEUE

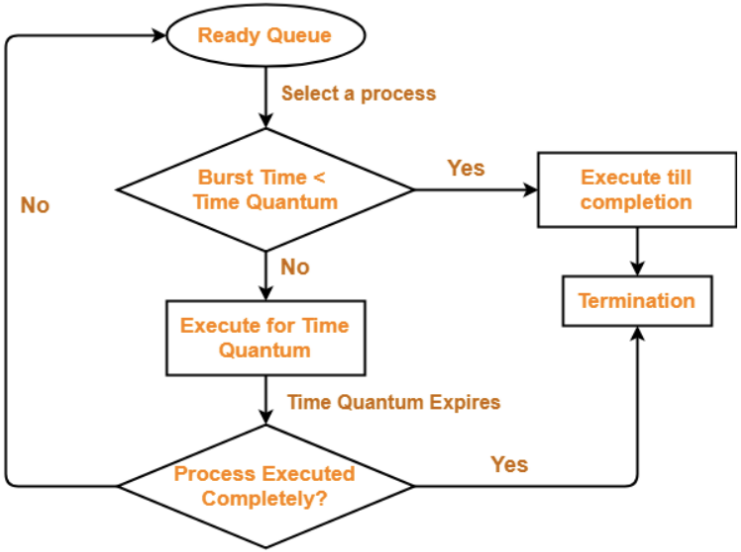
The circular queue is the core data structure in Round Robin Scheduling. It is used to maintain a queue of processes that need CPU time. In a circular queue, when the queue reaches its end, it loops back to the beginning, ensuring that processes are revisited in a circular order.

Usage:

- When a process gets its quantum and finishes or is paused, it is placed back at the end of the queue (or skipped if it finishes).
- This ensures that all processes get equal CPU time, making it ideal for time-sharing systems.

Algorithm Explanation

- **Initialization:** A circular queue holds processes ready for execution, each with attributes like remaining execution time, process ID, etc.
- **Time Quantum:** A fixed time quantum (time slice) is defined, limiting how long a process can run before switching to the next.
- **Process Enqueue:** New processes entering the ready state are added to the circular queue.
- **Execution:** The scheduler dequeues a process to run for a time quantum. If it finishes before the quantum expires, it's removed from the queue.
- **Time Slice Expiry:** If the process doesn't finish, its remaining time is updated, and it's re-enqueued at the end.
- **Next Process:** The scheduler repeats the process until all are completed.
- **New Arrivals:** New processes arriving during execution are added to the end of the queue.

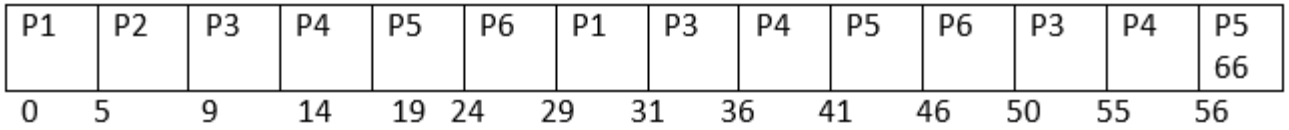


Example

Process ID	Arrival Time	Burst Time
P1	0	7
P2	1	4
P3	2	15
P4	3	11
P5	4	20
P6	4	9

Completion Time	Turn Around Time	Waiting Time
31	31	24
9	8	4
55	53	38
56	53	42
66	62	42
50	46	37

Time Quantum: 5



Code

```c
#include <stdio.h>
#include <stdlib.h>

#define MAX_PROCESSES 100 // Maximum number of processes

// Structure for each process
struct Process {
    int pid;          // Process ID
    int arrivalTime;    // Arrival time of the process
    int burstTime;     // Burst time of the process
    int remainingTime;  // Remaining burst time of the process
    int waitingTime;    // Waiting time of the process
    int turnaroundTime; // Turnaround time of the process
};

// Function to simulate round-robin scheduling and store results
void roundRobin(struct Process processes[], int totalProcesses, int timeSlice) {
    int timeElapsed = 0;  // Tracks the total time elapsed
    int completedProcesses = 0;

    // Initialize waiting time and turnaround time for each process
    for (int i = 0; i < totalProcesses; i++) {
        processes[i].remainingTime = processes[i].burstTime; // Initialize remaining time
        processes[i].waitingTime = 0; // Initialize waiting time
        processes[i].turnaroundTime = 0; // Initialize turnaround time
    }

    printf("Executing processes with a time slice of %d units:\n", timeSlice);

    // Loop until all processes are completed
    while (completedProcesses < totalProcesses) {
        for (int i = 0; i < totalProcesses; i++) {
            if (processes[i].remainingTime > 0 && processes[i].arrivalTime <= timeElapsed) {
                // Determine execution time
                int executedTime = (processes[i].remainingTime > timeSlice) ? timeSlice : processes[i].remainingTime;

                // Update time elapsed and remaining time
                timeElapsed += executedTime;
                processes[i].remainingTime -= executedTime;

                if (processes[i].remainingTime == 0) {
                    // Process has completed
                    completedProcesses++;
                    processes[i].turnaroundTime = timeElapsed - processes[i].arrivalTime; // Calculate turnaround time
                    processes[i].waitingTime = processes[i].turnaroundTime - processes[i].burstTime; // Calculate waiting time
                }
                printf("Process %d executed for %d units.\n", processes[i].pid, executedTime);
            }
        }
    }

    // Print results in a table format
    printf("\n--- Execution Results ---\n");
    printf("| Process ID | Waiting Time | Turnaround Time |\n");
    printf("|------------|--------------|-----------------|\n");
    for (int i = 0; i < totalProcesses; i++) {
        printf("| %11d | %12d | %15d |\n", processes[i].pid, processes[i].waitingTime, processes[i].turnaroundTime);
    }

    // Calculate and display average times
    int totalWaitingTime = 0, totalTurnaroundTime = 0;
    for (int i = 0; i < totalProcesses; i++) {
        totalWaitingTime += processes[i].waitingTime;
        totalTurnaroundTime += processes[i].turnaroundTime;
    }

    printf("\nAverage Waiting Time: %.2f\n", (float)totalWaitingTime / totalProcesses);
    printf("Average Turnaround Time: %.2f\n", (float)totalTurnaroundTime / totalProcesses);
}

// Function to print the process list
void displayProcesses(struct Process processes[], int totalProcesses) {
    if (totalProcesses == 0) {
        printf("No processes in the list.\n");
        return;
    }

    printf("Process List: \n");
    for (int i = 0; i < totalProcesses; i++) {
        printf("Process ID: %d, Arrival Time: %d, Burst Time: %d\n", processes[i].pid, processes[i].arrivalTime, processes[i].burstTime);
    }
}

// Main menu-driven program
int main() {
    struct Process processes[MAX_PROCESSES];
    int choice, totalProcesses = 0, pid, arrivalTime, burstTime, timeSlice;

    do {
        printf("\n--- Round Robin Scheduling Menu ---\n");
        printf("1. Add Process\n");
        printf("2. Display Processes\n");
        printf("3. Execute Round Robin Scheduling\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                if (totalProcesses >= MAX_PROCESSES) {
                    printf("Cannot add more processes. Maximum limit reached.\n");
                    break;
                }
                printf("Enter Process ID: ");
                scanf("%d", &pid);
                printf("Enter Arrival Time: ");
                scanf("%d", &arrivalTime);
                printf("Enter Burst Time: ");
                scanf("%d", &burstTime);
                processes[totalProcesses++] = (struct Process){pid, arrivalTime, burstTime, 0, 0, 0}; // Add new process
                printf("Process %d added successfully!\n", pid);
                break;
            case 2:
                displayProcesses(processes, totalProcesses);
                break;
            case 3:
                if (totalProcesses == 0) {
                    printf("No processes available to execute.\n");
                } else {
                    printf("Enter Time Slice: ");
                    scanf("%d", &timeSlice);
                    roundRobin(processes, totalProcesses, timeSlice);
                }
                break;
            case 4:
                printf("Exiting the program.\n");
                break;
            default:
                printf("Invalid choice. Please try again.\n");
        }
    } while (choice != 4);

    return 0;
}
```

Output

```
--- Round Robin Scheduling Menu ---
1. Add Process
2. Display Processes
3. Execute Round Robin Scheduling
4. Exit
Enter your choice: 2
Process List:
Process ID: 1, Arrival Time: 0, Burst Time: 10
Process ID: 2, Arrival Time: 2, Burst Time: 8
Process ID: 3, Arrival Time: 3, Burst Time: 12
```

```
--- Round Robin Scheduling Menu ---
1. Add Process
2. Display Processes
3. Execute Round Robin Scheduling
4. Exit
Enter your choice: 3
Enter Time Slice: 2
Executing processes with a time slice of 2 units:
Process 1 executed for 2 units.
Process 2 executed for 2 units.
Process 3 executed for 2 units.
Process 1 executed for 2 units.
Process 2 executed for 2 units.
Process 3 executed for 2 units.
Process 1 executed for 2 units.
Process 2 executed for 2 units.
Process 3 executed for 2 units.
Process 1 executed for 2 units.
Process 2 executed for 2 units.
Process 3 executed for 2 units.
Process 1 executed for 2 units.
Process 3 executed for 2 units.
Process 3 executed for 2 units.

--- Execution Results ---
| Process ID | Waiting Time | Turnaround Time |
|------------|--------------|-----------------|
|          1 |           16 |              26 |
|          2 |           12 |              20 |
|          3 |           15 |              27 |

Average Waiting Time: 14.33
Average Turnaround Time: 24.33
```

Conclusion And References

Conclusion: We implemented a Round Robin scheduling algorithm using a circular queue to effectively manage process execution in a simulated operating system environment. The user-friendly menu-driven interface enabled users to seamlessly add processes, display their details, and execute scheduling with customizable time slices. This design not only enhanced usability but also provided a clear visualization of the scheduling process. We successfully calculated and displayed key metrics such as waiting time and turnaround time, demonstrating the algorithm's ability to ensure fair CPU time-sharing among processes while minimizing response delays.

References:

- Reema Thareja - Data Structures using C, Oxford
- JavaTpoint: https://www.javatpoint.com/os-round-robin-scheduling-algorithm